DEVSECOPS / CLOUD SECURITY PORTFOLIO PROJECT

Deal Aggregator

A beginner's guide to the whole project

How it works from top to bottom - what every piece does and why it is there, how code travels from your laptop to the cloud, how it is protected, which phases are done, and what is left to build.

Inside this guide

#	What you will find
1	The project in one minute - and what DevSecOps / CloudSec mean
2	Words you need first (beginner glossary)
3	The big picture - every piece and why it exists
4	Top to bottom: what happens when someone opens the site
5	How deals get into the database (the scraper)
6	How code gets from your laptop to the cloud (CI/CD)
7	Watching the system (monitoring and logging)
8	How the project is protected (defense in depth)
9	Phases - what is done and what is left
10	Honest gaps right now
11	How to learn from this project + interview practice
12	Command cheat sheet

Status as of October 2026: Phases 1-6 done, Phase 7A (application & container hardening) done and deployed to EC2. Next: Phase 7B - Kubernetes hardening.

SECTION 1

The project in one minute

A website that shows Sri Lankan bank card discounts in one place - built so that you can learn to ship and protect real software in the cloud.

Banks in Sri Lanka run many card promotions ("20% off at Keells with HNB credit cards"). They are scattered across many websites. **Deal Aggregator** collects them automatically from **mypromo.lk**, stores them in a database, and shows them on one searchable website.

The website itself is deliberately simple. **The real point of the project is everything around the website**: how it is packaged into containers, tested automatically, shipped to a cloud server, watched while it runs, and protected from attackers. That "everything around it" is exactly what DevSecOps and Cloud Security engineers do every day.

What do these job titles actually mean?

- **DevOps** - automating how software is built, tested and deployed, so shipping is fast and boring.
- **DevSecOps** - the same, but security is built into *every* step (code, build, deploy, run) instead of being checked once at the end. People call this "shifting security left".
- **Cloud Security (CloudSec)** - protecting what runs in the cloud: servers, networks, identities (who can do what), secrets and data.

What the project demonstrates

Skill	Where you can see it in this project
Containers	Every part runs in Docker: frontend, backend, database, scraper, monitoring
CI/CD	GitHub Actions tests, scans, builds and deploys on every push
Cloud	Runs on an AWS EC2 server protected by a Security Group (firewall)
Observability	Prometheus + Grafana for metrics, Loki for logs
Security	Hardened API, non-root containers, database auth, closed ports, vulnerability scanning
Kubernetes	Manifests in the k8s/ folder - being hardened in the next phase

SECTION 2

Words you need first

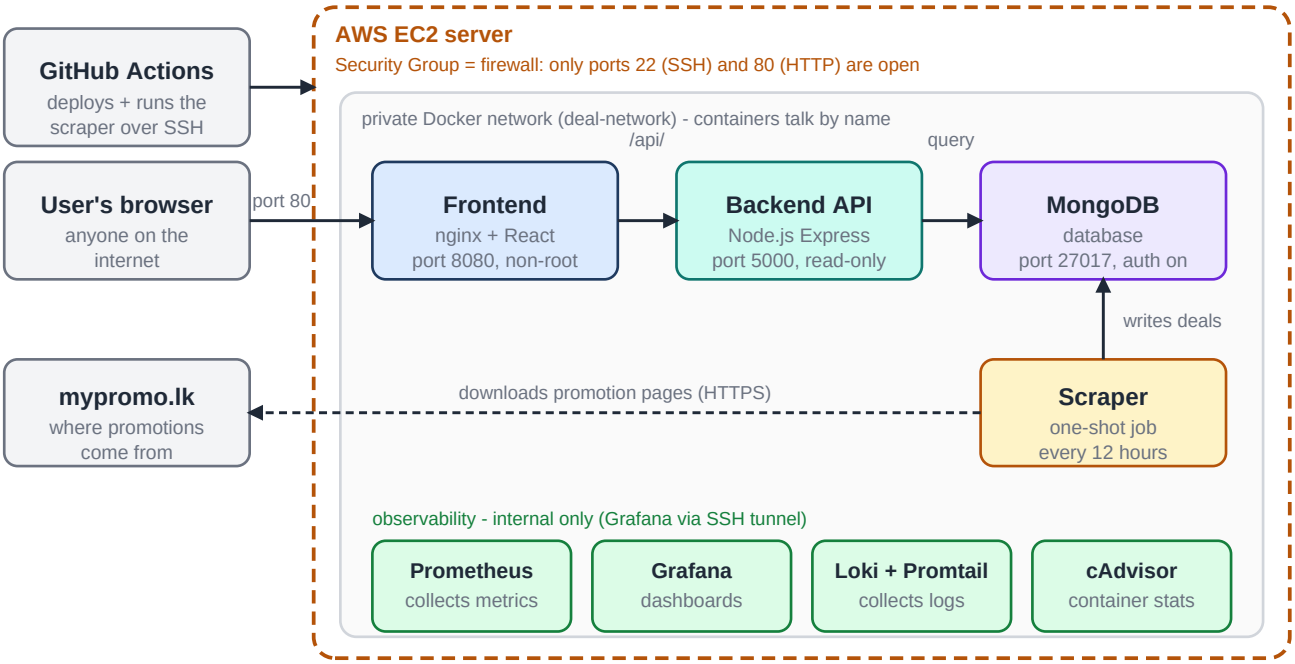
Read this once. Every later section uses these words. Come back to it whenever a term is fuzzy.

Term	Plain-English meaning	In this project
Server / EC2	A computer that is always on and reachable over the internet. EC2 is AWS's rent-a-server service.	Your EC2 instance runs the whole app.
IP address	The "phone number" of a computer on a network.	Your server's public IP, e.g. 13.60.92.239.
Port	A numbered door on a computer. One IP has 65,535 doors; each program listens on one.	80 = website, 22 = SSH, 5000 = API, 27017 = MongoDB.
Firewall / Security Group	Rules that decide which doors (ports) are open to whom. AWS calls it a Security Group.	Only 22 and 80 are open to the internet.
SSH	A secure way to log in to a remote server's terminal, using a key file.	How you - and GitHub Actions - get into EC2.
Container	A program packed together with everything it needs, running isolated from other programs. Like a shipping container for software.	Frontend, backend, DB, ... each in one.
Image	The frozen template a container is started from (the "recipe"). Built from a Dockerfile.	deal-aggregator-backend, -frontend, -scraper.
Docker / Compose	Docker runs containers. Docker Compose starts many containers together from one YAML file.	docker-compose.yml (laptop), docker-compose.prod.yml (EC2).
Registry	An online store for images.	Docker Hub (dockersula/...).
Volume	A folder that survives when a container is deleted - where data lives.	mongo-data keeps the deals; grafana-data keeps dashboards.
API	A program that answers requests from other programs with data (here: JSON).	GET /deals returns the deal list.
Reverse proxy	A server that sits in front of others and forwards requests to them.	nginx forwards /api/... to the backend.
Environment variable	A setting given to a program from outside its code.	MONGO_URI, PORT, CLIENT_ORIGIN.
Secret	A setting that must stay private: passwords, keys, tokens.	Mongo password, SSH key, Docker Hub token.
CI/CD	Continuous Integration / Continuous Deployment: robots that test and ship every change.	GitHub Actions workflows.
CVE	A publicly known security bug in some software, with an ID like CVE-2024-1234.	Trivy looks for CVEs in the images.
Kubernetes (K8s)	A system that runs containers across many servers, restarts them when they die and scales them up and down.	Next phase - the k8s/ folder.

SECTION 3

The big picture

Every box below is one container. The dashed orange line is your EC2 server; only the browser arrow comes in from the public internet.



The three tiers (that's why it's called a "3-tier app")

Tier	Container	Its one job
1. Presentation	Frontend (nginx + React)	Sends the website files to the browser. Also acts as the **gatekeeper**: it forwards requests starting with /api/ to the backend and blocks internal ones like /api/metrics.
2. Application	Backend (Node.js + Express)	Answers questions like "give me all active deals" by reading the database and replying with JSON. It is **read-only** - it can't create or delete deals.
3. Data	MongoDB	Stores the deals on a volume. Requires a username and password. Has **no public port** - only containers on the private network can reach it.

The helpers

- **Scraper** - a short-lived container that wakes up every 12 hours, downloads promotions from mypromo.lk, saves them to MongoDB and exits. It is the *only* thing that writes deals.
- **Monitoring & logging** - Prometheus, Grafana, cAdvisor, Loki and Promtail watch the other containers (Section 7).

> **Why split an app into tiers?**
>
> Each tier has one job and only talks to its neighbour. If an attacker breaks into the frontend, they still can't reach the database directly - it isn't on the internet and needs a password. This idea of building several walls instead of one is called **defense in depth**, and you will see it again and again.

SECTION 4

Top to bottom: someone opens the website

Follow one request through every layer. Being able to tell this story clearly is one of the best things you can do in an interview.

Step	Where	What happens	Security check on the way
1	Browser	A visitor types http://<your EC2 IP>. The browser sends an HTTP request to port 80.	-
2	AWS	The Security Group checks the door number. Port 80 is allowed, so the request gets in.	Any other port (27017, 9090, ...) is dropped silently.
3	Docker	Docker forwards the server's port 80 to the frontend container's port 8080.	nginx runs as a normal user, not root.
4	nginx	Sends back index.html plus the JavaScript and CSS of the React app.	Adds security headers: Content-Security-Policy, X-Frame-Options, nosniff. Hides its version.
5	Browser	React starts and asks for data: fetch('/api/deals?all=true') and fetch('/api/deals/meta').	CSP only lets scripts from your own site run.
6	nginx	Sees the /api/ prefix and forwards the request to backend:5000 over the private network, adding the visitor's real IP in X-Forwarded-For.	/api/metrics is answered with 404 - it never reaches the backend.
7	Express	The request passes the middleware chain: helmet, CORS, rate limiter, then the /deals route.	Max 300 requests per 15 min per IP. Input is escaped; only plain-text query values are accepted.
8	MongoDB	Express runs a query using the username and password inside MONGO_URI.	No password = no data. The DB is unreachable from outside.
9	Way back	MongoDB returns the deals -> Express turns them into JSON -> nginx passes it on -> the browser receives it.	-
10	Browser	React filters and sorts the deals and draws the cards on screen.	-

Two health checks - and why there are two

`/api/health` answers *"is the program alive?"*. `/api/ready` answers *"can it serve data right now?"* - it returns an error while MongoDB is unreachable. Kubernetes restarts a container whose health check fails, but only *stops sending traffic* to one that isn't ready. Restarting the API wouldn't fix a broken database, so the two must be separate.

SECTION 5

How deals get into the database

The website only reads. A separate, short-lived scraper does all the writing.

Step	What happens
1	Every 12 hours (or when you click *Run workflow*), GitHub runs `.github/workflows/scraper-cron.yml`.
2	It logs in to the EC2 server with SSH, using the private key stored in the GitHub secret `EC2_SSH_KEY`.
3	It runs `docker compose --profile scraper run --rm scraper`: start a temporary scraper container; --rm deletes it afterwards.
4	The scraper downloads about 10 category pages from mypromo.lk (supermarkets, dining, fashion, electronics, travel), retrying if the network fails.
5	It reads the HTML with a library called cheerio and pulls out the brand, discount, bank and dates, then guesses the offer type (cashback, % off, instalment) and card type.
6	It **upserts** each deal into MongoDB: update it if it already exists, insert it if it is new. Running the scraper twice therefore never creates duplicates.
7	It prints a summary and exits with code 0 (success) or 1 (failure), so GitHub marks the run green or red.

Why not let the website write deals?

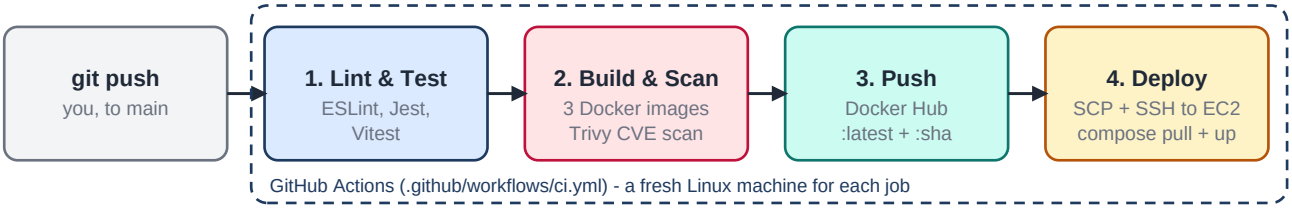
Before Phase 7A the API had public POST and DELETE endpoints, and anyone could wipe the database with one curl command - the "Admin" button only hid a button in the browser. Since only the scraper needs to write, the write endpoints were removed. An endpoint that doesn't exist can't be attacked. This principle is called **least functionality**.

In Kubernetes (next phase) the same scraper becomes a **CronJob** - Kubernetes' built-in timer for one-shot jobs - and the GitHub schedule is no longer needed.

SECTION 6

How code gets from your laptop to the cloud

You never copy files to the server by hand. You push to GitHub and robots do the rest.



Pull requests run steps 1-2 only. Pushes to main run all four. A failing step stops everything after it.

Job	What it does	Why it matters
1. Lint & Test	ESLint checks code style and common mistakes; Jest (backend) and Vitest (frontend) run the automated tests.	Bugs are caught minutes after you write them, not by users.
2. Build & Scan	Builds the 3 Docker images and scans them with **Trivy** for known vulnerabilities (CVEs) in the OS and npm packages.	You learn about vulnerable libraries before they reach production. (Currently report-only.)
3. Push	Uploads the images to Docker Hub, tagged :latest and with the Git commit ID.	The commit tag tells you exactly which code is running and makes rollbacks possible.
4. Deploy	Copies docker-compose.prod.yml and the monitoring configs to EC2 (SCP), then over SSH runs docker compose pull and up -d.	Same steps every time - no "works on my machine".

Where do the secrets live?

Secret	Stored in	Used for
DOCKERHUB_TOKEN	GitHub Secrets (encrypted)	Pushing images in job 3
EC2_SSH_KEY, EC2_HOST, EC2_USERNAME	GitHub Secrets	Deploy job + scraper schedule
MONGO_USERNAME / MONGO_PASSWORD, GRAFANA_PASSWORD	~/deal-aggregator/.env on EC2 (chmod 600)	Docker Compose fills them into the containers

Why did .env suddenly become necessary?

Before, there were no secrets at all: the database had no password and Grafana's password was "admin". Once MongoDB got a real password, it needed a home that is **not git** - your repo is public portfolio material and git history is forever. Docker Compose automatically reads a .env file next to the compose file, and the pipeline never copies or overwrites it. In Phase C the pipeline will write this file from GitHub Secrets so no manual step is left.

SECTION 7

Watching the system

You can't fix - or defend - what you can't see. Observability = metrics + logs (+ alerts later).

Tool	Job	How it works here
prom-client	Exposes numbers about the API	The backend serves /metrics: memory, CPU, event-loop lag, ...
cAdvisor	Measures every container	CPU, memory and network per container, read from Docker.
Prometheus	Collects and stores metrics	Every 15 s it pulls /metrics from the backend and cAdvisor (prometheus/prometheus.yml).
Promtail	Collects logs	Reads every container's console output from Docker and ships it to Loki.
Loki	Stores logs	Lets you search all logs in one place instead of SSH-ing in.
Grafana	Shows it all	Dashboards for metrics and logs. Bound to 127.0.0.1 - reach it with an SSH tunnel.

Opening Grafana safely

```
ssh -i <your-key>.pem -L 3001:localhost:3001 ubuntu@<EC2-IP>
# then open http://localhost:3001 in your browser (user: admin)
```

The **-L** flag builds a private tunnel: port 3001 on your laptop is connected to port 3001 on the server, through the encrypted SSH connection. Grafana is never on the public internet, but you can still use it.

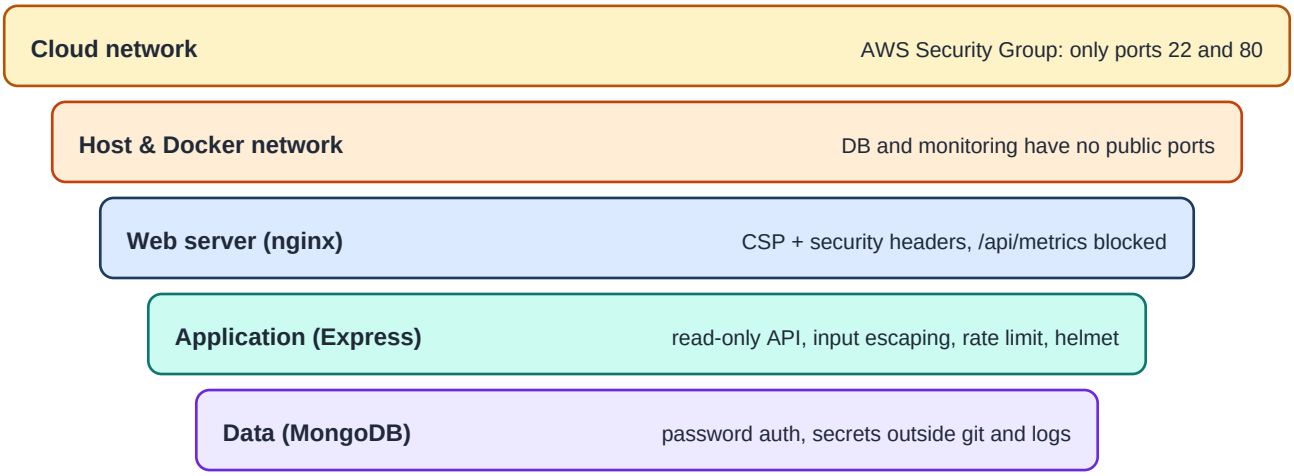
Why monitoring is a security tool

- A sudden spike of HTTP 429 ("too many requests") responses means someone is hammering your API.
- A container restarting again and again can mean a crash - or an exploit attempt.
- Logs are your evidence when something goes wrong: who, what, when. Detection and incident response start here.

SECTION 8

How the project is protected

No single control is perfect, so there are several walls. An attacker has to beat all of them.



Layer	Control	Attack it stops
Cloud network	Security Group allows only 22 and 80	Scanning and attacking internal tools (Prometheus, cAdvisor, the DB)
Docker network	No published ports for backend, DB, Prometheus, Loki, cAdvisor; Grafana on localhost only	Direct access if the firewall is ever misconfigured
nginx	Content-Security-Policy, X-Frame-Options, nosniff, server_tokens off, /api/metrics blocked	Cross-site scripting (XSS), clickjacking, information leaks
Express API	Read-only routes, regex escaping, string-only query params, 100-char search limit	Database wipe, ReDoS (freezing the server), NoSQL injection
Express API	Rate limit per real client IP, helmet headers, no X-Powered-By	Flooding / scraping, fingerprinting your stack
MongoDB	Username + password; credentials in .env, never in git or logs	Unauthorized data access, leaked passwords
Containers	Run as non-root; npm ci from lockfiles; pinned versions	Privilege escalation, surprise upgrades
Pipeline	Tests + Trivy scan before deploy; keys in GitHub Secrets	Shipping known-vulnerable code, leaked keys

Two real lessons from reviewing this project

- **Check that your claims are true.** The README said MongoDB had authentication - it didn't. The README said Trivy blocked builds - it was report-only. Reviewing your own work honestly is a core security skill.
- **Why version pinning matters.** The server was using mongo:latest and had silently upgraded to MongoDB 9.0. Pinned versions mean you decide when to upgrade, not the internet.

SECTION 9

Phases: what is done and what is left

The project is built in layers, like the skills of a DevSecOps engineer.

Phase	What was built	Status	What you learn
1	Core app: React + Express + MongoDB	DONE	3-tier architecture, REST APIs
2	Docker Compose, multi-stage builds	DONE	Images, containers, networks, volumes
3	CI: lint, test, Trivy, Docker Hub	DONE	GitHub Actions, shift-left testing
4	CD to EC2 + scheduled scraper	DONE	Automated deploys, SSH, CI secrets
5	Monitoring: Prometheus, Grafana, cAdvisor	DONE	Metrics and dashboards
6	Logging: Loki + Promtail	DONE	Centralized logs
7A	App & container hardening (Oct 2026)	DONE	AppSec basics, least privilege, secrets, attack surface
7B	Kubernetes hardening	NEXT	K8s security - core of the CKS certification
8 (C)	Secure CI/CD & supply chain	PLANNED	SAST, SCA, secret scanning, SBOM, image signing
9 (D)	GitOps (Argo CD) + in-cluster monitoring	PLANNED	GitOps, alerting, incident basics
E	Portfolio polish	PLANNED	Communicating your work: diagrams, threat model
F	Cloud security track on AWS	OPTIONAL	Terraform, IAM, OIDC, SSM, GuardDuty

What Phase 7A changed (already live on EC2)

- Removed the public POST/DELETE endpoints and the fake "Admin" button.
- Escaped user input (stops ReDoS and NoSQL injection); rate limit now per real visitor IP.
- Added /ready next to /health, and a graceful shutdown on SIGTERM.
- All containers run as non-root; nginx moved to port 8080; builds use npm ci.
- nginx security headers and CSP; /api/metrics blocked.
- MongoDB authentication with credentials in .env; 5 public ports closed; Grafana password required.
- Pinned versions: mongo 9.0.2, Prometheus 3.15.0, Grafana 13.2.3.
- Full details: docs/hardening/phase-a.md in the repo.

What is left - and what each phase teaches

Phase 7B - Kubernetes hardening (next)

- **MongoDB as a StatefulSet** with a password Secret - the right way to run databases in K8s.
- **securityContext + Pod Security Admission** - Kubernetes *refuses* to run containers that are root or can write to their own filesystem.
- **NetworkPolicies** - default-deny traffic inside the cluster: the frontend may talk to the backend, but never to the database. This stops *lateral movement*.
- **Ingress (+ TLS)** instead of a raw NodePort; CORS locked to the real hostname.
- **HPA, PodDisruptionBudget, ResourceQuota** - scaling, availability, and limits against resource abuse.
- **Sealed Secrets** - encrypted secrets that are safe to keep in git.

Phase C - Secure CI/CD and supply chain

- Parallel jobs; build each image once, scan it, and push *that exact* image.
- Trivy becomes **blocking**; add **gitleaks** (leaked secrets), **Semgrep** (code bugs), **npm audit**, and **Checkov** (misconfigured K8s YAML).
- **SBOM** (a list of everything inside an image) and **cosign** signatures (proof the image came from your pipeline).
- Pin GitHub Actions to exact versions; give the workflow minimal permissions; write .env from GitHub Secrets.

Phase D - GitOps and in-cluster observability

- **Argo CD** keeps the cluster identical to what's in git: every change is reviewed, logged and reversible.
- **kube-prometheus-stack** + alert rules (pod keeps restarting, scraper job failed).

Phase E - Portfolio polish

- Architecture diagram, screenshots (app, Grafana, Argo CD), a **threat model** table, and a README that matches the code exactly.

Phase F (optional) - the Cloud Security track

- **Terraform** for the EC2 instance, Security Group and IAM - infrastructure as code that can be reviewed and scanned.
- **GitHub OIDC to AWS** - no long-lived cloud keys stored anywhere.
- **AWS SSM Session Manager** instead of SSH - close port 22 completely.
- **CloudTrail + GuardDuty** - record every AWS API call and detect threats.
- **IAM least privilege** - the most important single skill in cloud security.

SECTION 10

Honest gaps right now

Every real system has known weaknesses. Knowing yours - and the plan to fix them - is a strength in an interview, not a weakness.

Gap	Why it matters	Fixed in
Site uses HTTP, not HTTPS	Traffic between the visitor and the server is not encrypted.	7B (Ingress + TLS), needs a domain name
SSH port 22 is open to the whole internet	GitHub Actions needs it today. It's protected by key-only login, but it's still an exposed door.	F (SSM Session Manager)
Trivy only reports	A badly vulnerable image could still be deployed.	C
MongoDB in Kubernetes has no password; CORS allows any origin	Only affects the K8s version (EC2 is already fixed).	7B
.env is created by hand on the server	Easy to forget when rebuilding the server.	C
Changing MONGO_PASSWORD needs a manual step	Mongo only reads it when the database is first created.	Documented; automated later
One server, one database	If EC2 goes down, the site goes down (no high availability).	7B / D (replicas in K8s)

SECTION 11

How to learn from this project

A suggested order for a CloudSec beginner, then hands-on exercises using your own project.

Order	Topic	Focus on
1	Linux & networking	Terminal basics, IP, ports, DNS, HTTP request/response, TLS
2	Docker	Images vs containers, Dockerfiles, volumes, networks, non-root users
3	Git & GitHub Actions	Branches, pull requests, workflow YAML, secrets
4	AWS fundamentals	EC2, VPC, Security Groups, **IAM** (users, roles, policies)
5	Security fundamentals	OWASP Top 10, least privilege, defense in depth, threat modeling
6	Kubernetes	Pods, Deployments, Services, then RBAC, NetworkPolicies, Pod Security
7	Infrastructure as Code	Terraform, scanning IaC for misconfigurations
8	Cloud security services	CloudTrail, GuardDuty, Security Hub, KMS, Secrets Manager

Hands-on exercises (do them on your own project)

#	Try this	What you should understand afterwards
1	`curl -I http://<EC2-IP>`	Read every response header and explain what it protects against.
2	`curl -m 5 http://<EC2-IP>:9090`	Why it times out: the Security Group drops the packet.
3	`docker compose exec backend whoami`	Prints "node", not "root" - and why that limits damage.
4	Stop MongoDB, then call /api/health and /api/ready	Health stays 200, ready turns 503 - liveness vs readiness.
5	Open a Trivy result in GitHub Actions	Pick one CVE, read it, and decide: real risk for this app or not?
6	Find your own visit in Loki (Grafana -> Explore)	How logs help you investigate an incident.
7	`curl -g "http://<EC2-IP>/api/deals?bank[$ne]=x"`	The attack is ignored - why string-only parameters matter.

Interview questions to practise with this project

- "Walk me through what happens when a user opens your site." (Section 4)
- "How do you handle secrets? Where does each one live?" (Section 6)
- "What's the difference between a liveness and a readiness probe?" (Section 4)
- "Why run containers as non-root?"
- "What security problems did you find when you reviewed your own project?" (Section 8)
- "How would you give CI access to a server without opening SSH to the internet?" (Phase F)
- "How would you stop an attacker in the frontend from reaching the database?" (Phase 7B NetworkPolicies)

Certifications that fit this path

- **AWS Cloud Practitioner -> AWS Security Specialty** - cloud security fundamentals to advanced.
- **CKA -> CKS** (Certified Kubernetes Security Specialist) - Phase 7B covers a large part of the CKS topics.
- Optional foundation: **CompTIA Security+** for general security vocabulary.

SECTION 12

Command cheat sheet

On your laptop

```
cp .env.example .env                    # create local credentials (first time)
docker compose up --build               # start everything locally
docker compose --profile scraper run --rm scraper   # fill the DB with deals
docker compose logs -f backend          # follow the backend's logs
docker compose down                     # stop everything (data is kept)
cd backend && npm test                  # run the backend tests
```

On the EC2 server

```
ssh -i <key>.pem ubuntu@<EC2-IP>
cd ~/deal-aggregator
docker compose -f docker-compose.prod.yml ps            # what is running?
docker compose -f docker-compose.prod.yml logs backend --tail 50
docker compose -f docker-compose.prod.yml restart backend
curl localhost/api/ready                                # is the app healthy?
```

Useful checks from anywhere

```
curl http://<EC2-IP>/api/health      # liveness  -> {"status":"ok"}
curl http://<EC2-IP>/api/ready       # readiness -> {"status":"ready","db":true}
curl -I http://<EC2-IP>              # see the security headers
```

Key files in the repository

File / folder	What it is
backend/src/server.js	Express app: middleware, health checks, graceful shutdown
backend/src/routes/deals.js	The read-only /deals API
frontend/nginx.conf	Web server, reverse proxy and security headers
scraper/scrape.js	The mypromo.lk scraper
*/Dockerfile	How each image is built
docker-compose.yml / .prod.yml	Local and production stacks
.github/workflows/	CI/CD pipeline and scraper schedule
k8s/	Kubernetes manifests (Phase 7B)
docs/hardening/phase-a.md	Detailed what & why of Phase 7A